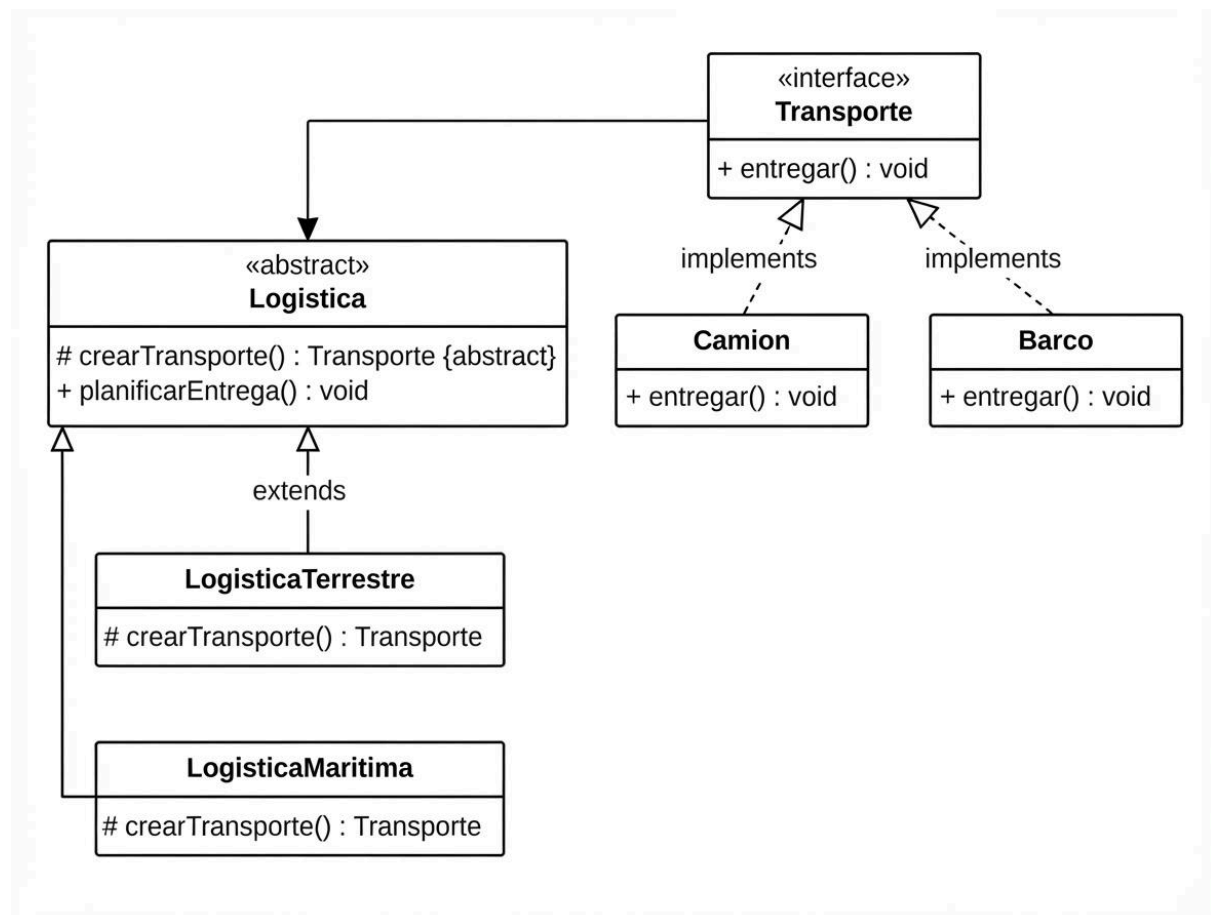


Trabajo Práctico: Patrones de Diseño

1. Diagrama UML de Clases:

a. Elaborar el diagrama UML de clases correspondiente al patrón Factory Method.



b. Identificar y describir las clases, interfaces, atributos, métodos y relaciones involucradas.

b. Identificación y descripción de clases, interfaces, atributos, métodos y relaciones

1. Interfaces y Clases

- **Transporte (Interfaz - Producto):**
 - Descripción: Define el contrato común que deben cumplir todos los medios de transporte.
 - Métodos: `entregar()` (método abstracto que ejecuta la acción de entrega).
- **Camion y Barco (Clases Concretas - Productos Concretos):**
 - Descripción: Implementan la interfaz **Transporte** con su lógica específica para tierra o mar.
 - Métodos: `entregar()` (imprime el mensaje correspondiente al tipo de transporte).
- **Logistica (Clase Abstracta - Creador):**
 - Descripción: Define el método fábrica abstracto y la lógica de negocio principal que utiliza el transporte.
 - Métodos:

- `crearTransporte()` (Método fábrica abstracto que obliga a las subclases a retornar un `Transporte`).
 - `planificarEntrega()` (Lógica de negocio que invoca a `crearTransporte()` y luego llama al método `entregar()`).
- `LogisticaTerrestre` y `LogisticaMaritima` (Clases Concretas - Creadores Concretos):
 - Descripción: Heredan de `Logistica` y sobrescriben el método fábrica para instanciar el producto correspondiente.
 - Métodos: `crearTransporte()` (retorna una nueva instancia de `Camion` o `Barco` respectivamente).

2. Relaciones involucradas

- Implementación (Realization): `Camion` y `Barco` implementan la interfaz `Transporte`.
- Herencia / Generalización (Inheritance): `LogisticaTerrestre` y `LogisticaMaritima` extienden (heredan de) la clase abstracta `Logistica`.
- Dependencia / Creación (Dependency): La clase creadora `Logistica` depende de la interfaz `Transporte` para poder operar con los objetos creados por sus subclases.

2. Análisis del Patrón Factory Method

a. Problema

Si creas objetos directamente usando el operador `new` en tu código principal (por ejemplo, `new Camión()`), tu sistema queda fuertemente acoplado. Si mañana necesitas agregar un nuevo transporte (como un avión), tendrías que modificar el código existente en varios lugares, lo que rompe las buenas prácticas y hace que el mantenimiento sea difícil y propenso a errores.

b. Solución

El patrón **Factory Method** propone delegar la creación de objetos a un método especializado (la "fábrica") dentro de clases abstractas.

- En lugar de instanciar un camión directamente, la lógica de negocio llama a un método abstracto `crearTransporte()`.
- Las subclases (`LogisticaTerrestre`, `LogisticaMaritima`) deciden qué objeto específico (`Camión` o `Barco`) crear. De esta forma, la lógica principal funciona con interfaces generales (`Transporte`, `Logistica`) sin importarle qué clase concreta está operando.

c. Consecuencias

- **Ventajas:**
 - Evita el acoplamiento directo entre clases.
 - Facilita la escalabilidad: puedes agregar nuevos transportes creando nuevas clases sin tocar el código que ya funciona (Principio Abierto/Cerrado).

- El código de creación se centraliza en un solo lugar.
- **Desventajas:**
 - El proyecto se vuelve más complejo porque obliga a crear muchas clases pequeñas e interfaces adicionales.

3. Implementación

a. Desarrollar un ejemplo de implementación en código para cada patrón de diseño.

Desarrollado en GitHub.

b. Incluir comentarios y una breve explicación del funcionamiento de la solución implementada.

b. Explicación:

El patrón Factory Method es un patrón de diseño creacional que resuelve el problema de crear objetos sin especificar sus clases concretas. En lugar de eso, delega la responsabilidad de la creación en las subclases.

1. Desacoplamiento de la Creación: La clase principal Logistica define un método abstracto llamado crearTransporte() (el "método fábrica") y un flujo de negocio en planificarEntrega(). Logistica sabe *qué* hacer con el transporte (entregar()), pero no sabe (ni le importa) si se trata de un Camion o de un Barco.
2. Extensibilidad (Principio Abierto/Cerrado): Si en el futuro necesitas agregar un nuevo medio de transporte (por ejemplo, un Avion), solo debes crear la clase que implemente Transporte y una nueva subclase creadora (LogisticaAerea), sin modificar el código existente de la lógica de negocio central.
3. Polimorfismo: El cliente interactúa únicamente con las abstracciones (Logistica y Transporte), lo que hace que el sistema sea altamente flexible y fácil de mantener.